

Guia de Padronização de Código

Este documento estabelece as regras básicas de padronização para o projeto **api-matriculas** em Go (Golang). Seguir estas diretrizes garantirá que o código do grupo seja legível, consistente e profissional.

1. Idioma do Código

A padronização do idioma é estrita e obrigatória para manter a consistência do projeto:

- **Código Fonte:** Utilize **somente o Inglês** para nomes de arquivos, variáveis, funções e structs.
- **Logs e Mensagens:** Comentários explicativos, mensagens de erro e logs de execução direcionados ao cliente ou sistema **devem ser escritos em Português (pt-BR)**.

2. Estrutura de Arquivos e Pastas

A linguagem Go possui convenções bem estabelecidas para o sistema de arquivos que devem ser seguidas rigorosamente:

- **Nome de Pastas (Pacotes):** Tudo em minúsculo, palavra única, sem hifens ou underlines se possível. Exemplo: `handlers`, `models`, `middleware`, `auth`.
- **Nome de Arquivos:** Sempre em minúsculo, utilizando `snake_case` (separado por underline) caso haja mais de uma palavra.
 -  **Certo:** `general_handle.go`, `admin_routes.go`, `user.go`
 -  **Errado:** `userHandler.go`, `AdminRoutes.go`

3. Nomenclatura no Código (Variáveis, Funções e Tipos)

3.1 Variáveis Locais e Parâmetros

Utilize `camelCase`. Para variáveis muito curtas ou contadores de loop, nomes de uma letra são aceitáveis no Go (ex: `i`, `r`, `w`).

-  **Certo:** `tokenData`, `secretKey`, `req`
-  **Errado:** `Token_Data`, `Secret_key`

3.2 Funções e Structs (Visibilidade)

Em Go, a primeira letra define a visibilidade da função ou variável para outros pacotes.

- **Exportadas (Públicas):** `PascalCase`. A primeira letra é **Maiúscula**. Use para tudo que precisa ser acessado de outro arquivo/pacote.

Exemplo: `GenerateStudentToken`, `LoginHandler`, `GeneralRoutes`, struct `StudentAuth`.

- **Não-Exportadas (Privadas):** `camelCase`. A primeira letra é **minúscula**. Use para funções auxiliares internas do mesmo pacote.

Exemplo: `gerarTokenInterno`, `checkUser`.

3.3 Estruturas (Structs) e JSON Tags

Os nomes dos campos na Struct devem seguir a regra de visibilidade (geralmente `PascalCase` para serem convertidos em JSON). Já as tags JSON seguem o padrão REST (`snake_case`).

```
type User struct {
    UserID    string `json:"user_id"`
    Role      string `json:"role"`
    CreatedAt string `json:"created_at"`
}
```

4. Resumo Rápido das Regras

- **Idioma Base:** Inglês estrito para código; Português para logs/erros.
- **Pastas:** minúsculo (`models`)
- **Arquivos:** `snake_case.go` (`user_handler.go`)
- **Variáveis:** `camelCase` (`tokenData`)
- **Structs e Interfaces:** `PascalCase` (`StudentToken`)
- **Funções Exportadas:** `PascalCase` (`LoginHandler`)
- **Funções Internas:** `camelCase` (`checkUser`)
- **JSON Tags:** `snake_case` (``json:"institution_id"``)